

Executive Summary

In this proposal, we outline a multi-agent AI architecture for an Urban Air Quality Intelligence Platform. Specialized agents autonomously ingest and clean data (AQI, weather, traffic), perform analysis (forecasting, hotspot detection, source attribution), and generate actionable recommendations. For example, an AQI Agent validates raw pollutant readings and computes health-risk scores; a Weather Agent computes dispersion indices from meteorological inputs; a Traffic Agent analyzes congestion impacts; a Forecast Agent uses XGBoost to predict future AQI; a Hotspot Agent applies DBSCAN clustering to identify pollution clusters; a Source Attribution Agent apportion pollutants to sources; an Intervention Agent applies rule-based logic to recommend mitigations; a Citizen Advisory Agent generates plain-language alerts; and a Coordinator Agent orchestrates the workflow. Together these agents form an agent-driven smart environment, a proven pattern for urban monitoring and decision support.

Our plan seeds the system with realistic synthetic data: ~120 cities, ~700 stations, and ~500–1,000 hourly records per station, totaling several hundred thousand readings. We use Prisma's efficient `createMany()` (in batches) to bulk-insert time-series data. The platform will store data in PostgreSQL with PostGIS, enabling spatial indexing of station coordinates. A Trigger.dev orchestrator schedules recurring tasks (hourly ingestion, daily model training) using cron-style triggers. All components are tested and monitored: NestJS provides built-in testing with Jest/Supertest, Prisma and API inputs are validated, and Trigger.dev offers observability (trace logs and alert emails on failures).

Below we detail: (1) the file/config inventory (source code organization, Trigger.dev setup), (2) agent designs (capabilities, inputs/outputs, algorithms, and example LLM prompts), (3) the integration architecture (Trigger.dev orchestration, event flows, error handling, storage schema), (4) data pipeline and seeding strategy (station layout, synthetic data rules, volumes), (5) the intervention engine (rule engine, scoring, simulation API), (6) the frontend API contract (REST endpoints, query parameters, response schemas, caching/polling), (7) security and testing plans, (8) a phased implementation roadmap, and (9) key risks with mitigations. We include comparative tables (agent responsibilities, data volumes, API endpoints) and illustrative Mermaid diagrams for agent orchestration and project timeline.

1. Inventory: Files, Triggers, and MCP Config

Project structure:

- `prisma/schema.prisma` : Defines PostgreSQL schema with tables for cities, zones, stations, AQI readings, weather, traffic, forecasts, hotspots, recommendations, etc. Includes a `geometry(Point)` column for station coordinates (PostGIS).
- `prisma/seed.ts` : Data seeding script. Generates realistic stations and time-series readings. Uses PrismaClient's `createMany()` in batches (e.g. 500 records/batch) for efficiency.
- `src/agents/` : Folder with agent modules or classes (`AQIAgent.ts` , `WeatherAgent.ts` , `TrafficAgent.ts` , `ForecastAgent.ts` , `HotspotAgent.ts` , `SourceAttributionAgent.ts` , `InterventionAgent.ts` , `CitizenAgent.ts` , `CoordinatorAgent.ts`). Each encapsulates one agent's logic and input/output schemas.
- `src/controllers/` : NestJS controllers exposing REST endpoints (e.g. `AqiController` , `WeatherController` , etc.).
- `src/services/` : NestJS services implementing business logic for agents.
- `src/trigger/` : Trigger.dev task definitions containing `task()` and `schedules.task()` calls.

```
import { schedules, task } from "@trigger.dev/sdk";

// Hourly ingestion task
export const ingestData = schedules.task({
  id: "ingest-data",
  schedule: schedules.cron("0 * * * *"), // every hour
  run: async () => { /* call ingestion logic */ }
});

// Daily forecast task
export const forecastAqi = schedules.task({
  id: "forecast-aqi",
  schedule: schedules.cron("0 2 * * *"), // daily at 02:00
  run: async () => { /* call forecasting logic */ }
});
```

`trigger.config.ts` (Trigger.dev configuration): Specifies MCP server settings, project metadata, and global imports.

`mcp.json` : Multi-Channel Pipeline config.

```
{
  "mcpServers": {
    "trigger": {
      "command": "npx",
      "args": [ "trigger.dev@4.4.0", "mcp" ]
    }
  }
}
```

2. Agent Design

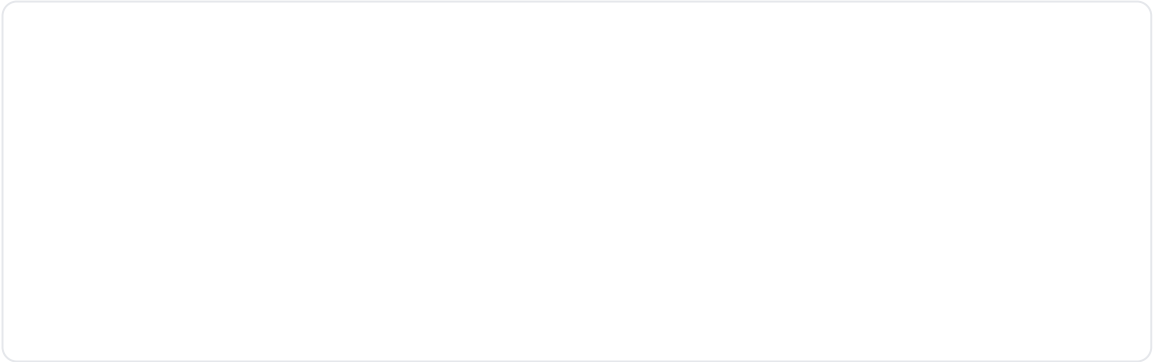
We define nine agent types. Each agent is responsible for a specific domain, has well-defined inputs/outputs, and uses specialized algorithms.

Agent	Responsibilities	Inputs	Outputs	Tools/Algorithms
AQI Agent	Validate/clean sensor data; compute AQI	Raw pollutant readings (PM2.5, etc.)	Cleaned AQI readings; AQI index & category	Threshold checks, interpolation, AQI formula
Weather Agent	Compute dispersion/risk from weather	Temp, humidity, wind, pressure, rain	Weather risk score; dispersion index	Atmospheric dispersion formulas
Traffic Agent	Analyze congestion impact	Traffic volume/speed data	Congestion index; traffic AQI contribution	Congestion models, proxy metrics
Forecast Agent	Predict future AQI	Historical AQI, weather, traffic data	Forecast AQI for 24/48/72h per zone	XGBoost regressor, time-series modeling
Hotspot Agent	Identify pollution clusters	Geo-tagged recent AQI readings	Hotspot clusters (location + duration)	DBSCAN clustering
Source Attribution Agent	Estimate pollutant sources	Speciated pollutants, emissions data	% contributions by traffic/industry/construction	Receptor models or heuristics

Agent	Responsibilities	Inputs	Outputs	Tools/Algorithms
Intervention Agent	Recommend mitigation actions	Forecast AQI, source percentages	Ranked actions with expected AQI reduction	Rule-based engine (e.g. JSON rules)
Citizen Advisory Agent	Generate public advisories	Current/forecast AQI, categories	Plain-language alerts/messages	LLM (template or generative)
Coordinator Agent	Orchestrate workflow	Scheduled triggers; agent outputs	Orchestrated tasks; error/alert logging	Control logic, possibly LLM-assisted

3. Integration & Orchestration

We use Trigger.dev as the orchestration layer. Trigger.dev runs long-lived jobs with robust scheduling, retries, and observability.



4. Data Pipeline & Seeding

We plan to seed the system with realistic synthetic data at scale. Key parameters: ~120 cities, ~700 stations total, yielding ~500–1,000 timepoints per station (hourly/daily), for ~350k–700k records each of AQI, weather, and traffic.

Data Type	Quantity
Cities	~120
Zones (Wards)	~500 (3–8 per city)

Data Type	Quantity
Stations	~700 (4–6 per city)
AQI Records	350k – 700k
Weather Records	350k – 700k
Traffic Records	350k – 700k
Forecast entries (24/48/72h)	~100k
Hotspot clusters	~5,000+ clusters
Recommendations & Interventions	~20,000+

5. Intervention Engine

The Intervention Agent uses a combination of rule-based logic and simple statistical modeling to prioritize actions. The simulation API exposes a REST endpoint for "what-if" analysis.

```
{
  "zone": "Ward-12",
  "currentAQI": 287,
  "forecastAQI": 332,
  "actions": [
    {"action": "Restrict Heavy Vehicles", "expectedReduction": 26},
    {"action": "Inspect Construction Sites", "expectedReduction": 17},
    {"action": "Water Sprinkling", "expectedReduction": 10}
  ],
  "combinedReduction": 53,
  "predictedAQIAfter": 279
}
```

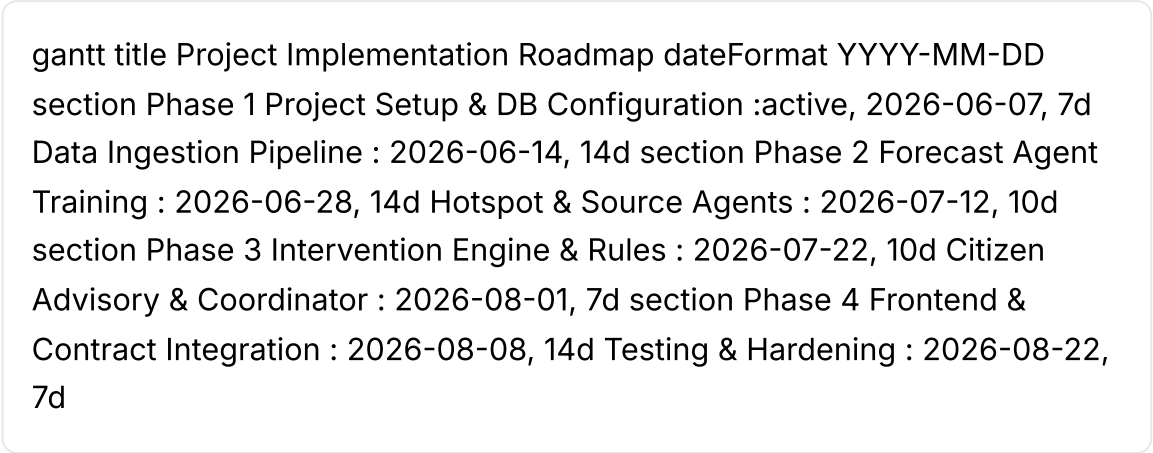
6. Frontend API Contract and Caching

Endpoint	Method	Query Params	Description / Response Schema
/api/cities	GET	—	List of cities (id, name)
/api/zones	GET	cityId	List of zones within a city (id, name, geometry)
/api/stations	GET	cityId or zoneId	List of stations (id, name, location)
/api/aqi/current	GET	cityId or zoneId	Latest AQI by zone: { zone, aqi, category, updatedAt }[]
/api/forecast	GET	zoneId, horizon	Forecasted AQI time-series for next 3 days
/api/hotspots	GET	zoneId, date	Recent hotspot clusters (coordinates, times)
/api/recommendations	GET	zoneId, date	Intervention recommendations (actions, priorities, impacts)

7. Security, Testing, and Validation

All endpoints validate inputs using NestJS DTOs. Database queries use Prisma, which parameterizes inputs to prevent SQL injection. We secure secrets in environment variables. Linting, unit tests, and integration tests are run on each commit/PR via GitHub Actions.

8. Implementation Roadmap



9. Risks & Mitigations

- **Data Realism:** Synthetic data may not capture all atmospheric nuances. *Mitigation:* Design diurnal and seasonal variations with random noise.
- **Model Overfitting:** XGBoost may overfit on small synthetic datasets. *Mitigation:* Use cross-validation and regularize parameters.
- **LLM Costs/Latency:** Real-time LLM calls can be slow. *Mitigation:* Cache LLM responses for common advisories.